

Sztuczna Inteligencja w grach

Porównanie metod sterowania agentami w grze Tetris

Mikołaj Nowak

9 maja 2026

Spis treści

1	Cel projektu	2
2	Opis środowiska i sposobu reprezentacji problemu	2
2.1	Mechanika gry	2
2.2	Stan i akcje	3
3	Zaimplementowane podejścia	3
3.1	Agent losowy	3
3.2	Agent heurystyczny	3
3.3	Metody uczenia ze wzmocnieniem	4
3.3.1	DQN	4
3.3.2	REINFORCE	4
3.3.3	A2C	4
3.3.4	PPO	4
3.4	Metody optymalizacyjne	4
3.4.1	Genetic Algorithm	4
3.4.2	Evolution Strategies	5
4	Metodyka eksperymentów	5
4.1	Trening / optymalizacja	5
4.2	Ewaluacja końcowa	5
5	Wyniki ilościowe	5
5.1	Postęp uczenia w czasie	5
5.2	Ewaluacja końcowa po 10 grach	8
5.3	Porównanie trzech głównych podejść	9
6	Wyniki jakościowe	9
6.1	Random	9
6.2	Heuristic	9
6.3	DQN	9
6.4	REINFORCE i PPO	10
6.5	A2C	10
6.6	GA i ES	10
7	Porównanie pracochłonności	10
8	Wnioski końcowe	10

1 Cel projektu

Celem projektu było przygotowanie prostej gry z agentami sterowanymi sztuczną inteligencją oraz porównanie trzech klas podejść do budowy logiki sterowania:

1. metod heurystycznych, czyli reguł zaprojektowanych ręcznie,
2. metod uczenia ze wzmocnieniem,
3. metod optymalizacyjnych / ewolucyjnych.

Jako środowisko wybrano grę **Tetris** ze względu na to, że:

- każdą rozgrywkę można ocenić jedną liczbą – końcowym wynikiem punktowym,
- przestrzeń akcji jest skończona i łatwa do wygenerowania,
- można porównywać zarówno skuteczność ilościową, jak i jakość zachowania bota,
- istnieje prosty tryb dla człowieka, dzięki czemu można sprawdzić działanie mechaniki gry także ręcznie.
- środowisko graficzne tetrisa nie wymaga skomplikowanych obliczeń, przez co jest łatwe do zaimplementowania

W projekcie przyjęto **bezwzględną** miarę jakości zachowania agenta: końcowy wynik punktowy w jednej grze. Tetris jest tu środowiskiem jednoosobowym, więc ocena nie wynika z jednoczesnej interakcji między wieloma botami, lecz z jakości gry pojedynczego agenta na tej samej planszy i przy tych samych zasadach.

2 Opis środowiska i sposobu reprezentacji problemu

2.1 Mechanika gry

Za środowisko posłużył Tetris na planszy prostokątnej. Agent otrzymuje kolejne klocki i musi zdecydować, jak je ułożyć, aby:

- czyścić linie,
- unikać tworzenia dziur,
- utrzymywać możliwie płaski profil planszy,
- nie doprowadzić do przegranej przez zbyt wysokie ułożenie klocków.

Funkcja nagrody została oparta na klasycznej punktacji Tetrisa i zależy od liczby linii usuniętych w jednym ruchu. Przyjęto następujące wartości: 0, 40, 100, 300 oraz 1200 punktów odpowiednio dla 0, 1, 2, 3 i 4 usuniętych linii. Oznacza to, że środowisko silnie premiuje ruchy prowadzące do jednoczesnego czyszczenia wielu linii, szczególnie czterech naraz. Za samo postawienie klocka bez wyczyszczenia linii nie jest przyznawana dodatkowa nagroda. Po zakończeniu gry od końcowego wyniku odejmowane są 2 punkty.

W eksperymentach jako miarę jakości agenta wykorzystano końcowy **score** uzyskany w całej rozgrywce.

Oprócz trybu botów przygotowano także własny tryb gry dla człowieka (`play_human.py`), sterowany klawiszami klawiatury.

2.2 Stan i akcje

Istotną cechą tej implementacji jest to, że agent nie wybiera pojedynczego ruchu w czasie rzeczywistym typu *przesuń w lewo o jeden* lub *obróć teraz*, tylko wybiera **jedną z wszystkich legalnych końcowych pozycji bieżącego klocka**.

Dla każdej legalnej akcji generowany jest stan wynikowy po opuszczeniu klocka. Opis stanu ma postać czterech cech:

1. liczba wyczyszczonych linii po ruchu,
2. liczba dziur na planszy,
3. nierówność profilu planszy (*bumpiness*),
4. łączna wysokość kolumn.

Takie podejście upraszcza problem i pozwala wszystkim agentom porównywać legalne ruchy w tej samej reprezentacji.

3 Zaimplementowane podejścia

3.1 Agent losowy

Agent *Random* wybiera losowo jedną z legalnych akcji. Nie uczy się i służy wyłącznie jako dolna linia odniesienia.

3.2 Agent heurystyczny

Agent heurystyczny został zaprojektowany ręcznie. Dla każdej możliwej akcji symulowany jest wynikowy stan planszy, a następnie liczona jest funkcja oceny:

$$\text{score}_H = 120 \cdot L - 45 \cdot H - 3 \cdot B - 2 \cdot T - 8 \cdot M + 12 \cdot W - 20 \cdot C,$$

gdzie:

- L – liczba linii usuniętych po ruchu,
- H – liczba dziur,
- B – nierówność sąsiednich kolumn,
- T – suma wysokości kolumn,
- M – maksymalna wysokość kolumny,
- W – głębokość *lewej studni*,
- C – kara za zablokowanie lewej kolumny.

Najważniejszą własną ideą heurystyki było **utrzymywanie lewej studni**. Agent preferuje sytuacje, w których pierwsza kolumna pozostaje relatywnie niższa od drugiej, ponieważ zwiększa to szansę na sensowne wykorzystanie długiego klocka. Jednocześnie karane jest zapełnianie lewej kolumny wysoko na planszy, bo wtedy studnia przestaje pełnić swoją funkcję.

W praktyce oznacza to strategię:

- czyścić linie, jeśli to możliwe,
- bardzo unikaj dziur,
- nie buduj zbyt wysokich struktur,
- zostaw pewne miejsce na długi klocek po lewej stronie.

W przypadku remisu między kilkoma ruchami agent losuje jeden z najlepszych.

3.3 Metody uczenia ze wzmocnieniem

W projekcie zaimplementowano cztery warianty agentów RL.

3.3.1 DQN

DQN jest metodą opartą o aproksymację wartości. Sieć neuronowa ocenia, jak dobry jest stan planszy po wykonaniu danego ruchu. Podczas treningu agent stosuje eksplorację typu ϵ -greedy, a podczas testów wybiera akcję o najwyższej przewidywanej wartości.

W tym projekcie DQN okazał się **najsilniejszym agentem** spośród wszystkich rozpatrywanych metod.

3.3.2 REINFORCE

REINFORCE jest klasyczną metodą policy gradient. Dla wszystkich legalnych ruchów sieć oblicza wynik, a po zastosowaniu funkcji softmax agent losuje akcję zgodnie z rozkładem prawdopodobieństwa. Po zakończeniu epizodu aktualizuje politykę tak, aby zwiększać prawdopodobieństwo ruchów prowadzących do wysokiego zwrotu.

3.3.3 A2C

A2C (Advantage Actor-Critic) wykorzystuje dwa elementy:

- **aktora**, który odpowiada za wybór ruchu,
- **krytyka**, który ocenia wartość stanu.

W teorii metoda ta powinna stabilizować uczenie względem prostszego REINFORCE, jednak w przeprowadzonych eksperymentach wypadła wyraźnie słabiej od pozostałych agentów RL.

3.3.4 PPO

PPO (Proximal Policy Optimization) jest rozwinięciem metod actor-critic. Kluczową ideą jest ograniczenie wielkości pojedynczej aktualizacji polityki, co zwiększa stabilność uczenia. W praktyce PPO dał wyraźnie lepsze wyniki niż A2C i REINFORCE, ale nadal był dużo słabszy od DQN.

3.4 Metody optymalizacyjne

W projekcie zaimplementowano dwie metody optymalizacji parametrów sieci.

3.4.1 Genetic Algorithm

Algorytm genetyczny operuje na populacji osobników, gdzie osobnik reprezentuje zestaw wag sieci oceniającej ruchy. W kolejnych generacjach stosowane są:

- selekcja,
- krzyżowanie,
- mutacja,
- elitaryzm.

Jakość osobnika jest mierzona wynikiem gry. W finalnych eksperymentach GA okazał się najlepszą metodą optymalizacyjną pod względem średniego wyniku.

3.4.2 Evolution Strategies

W Evolution Strategies nie utrzymuje się klasycznego krzyżowania populacji, tylko centralną politykę, do której dodawane są zaburzenia losowe. Na podstawie jakości zaburzonych wariantów aktualizowane są parametry centralnej polityki. Metoda ta dała dobre wyniki, ale w końcowej ewaluacji była słabsza od algorytmu genetycznego.

4 Metodyka eksperymentów

4.1 Trening / optymalizacja

Każdej metodzie przydzielono **30 minut budżetu czasowego**. Dotyczyło to zarówno:

- agentów RL i optymalizacyjnych w trakcie treningu,
- agentów nieuczących się (Random i Heuristic) w benchmarku czasowym, aby uzyskać porównywalne wykresy postępu najlepszego wyniku w czasie.

Podczas treningu co minutę zapisywano najlepszy wynik osiągnięty do tej pory. Na tej podstawie powstały wykresy postępu uczenia.

4.2 Ewaluacja końcowa

Po zakończeniu treningu dla każdego agenta wykonano **10 pełnych gier testowych**. W skrypcie ewaluacyjnym nie podawano ręcznie limitu `max_pieces`, więc użyta została bardzo duża wartość z konfiguracji projektu, co w praktyce pozwalało agentom grać aż do naturalnego zakończenia rozgrywki.

Wyniki z tych 10 gier zapisano do pliku CSV, a następnie obliczono:

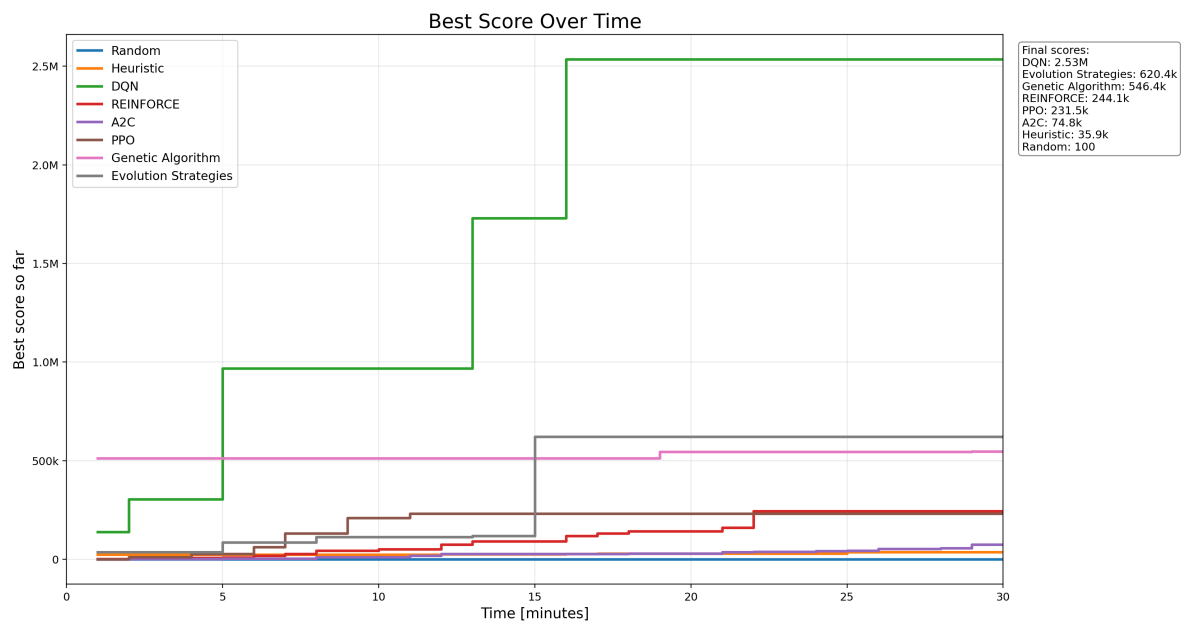
- średni wynik,
- medianę,
- odchylenie standardowe,
- najlepszy i najgorszy wynik,
- średnią liczbę usuniętych linii.

Dodatkowo dla każdego agenta zapisywano GIF najlepszej rozgrywki, ale z mniejszym domyślnym limitem liczby klocków, tak aby zapis wideo miał rozsądny rozmiar.

5 Wyniki ilościowe

5.1 Postęp uczenia w czasie

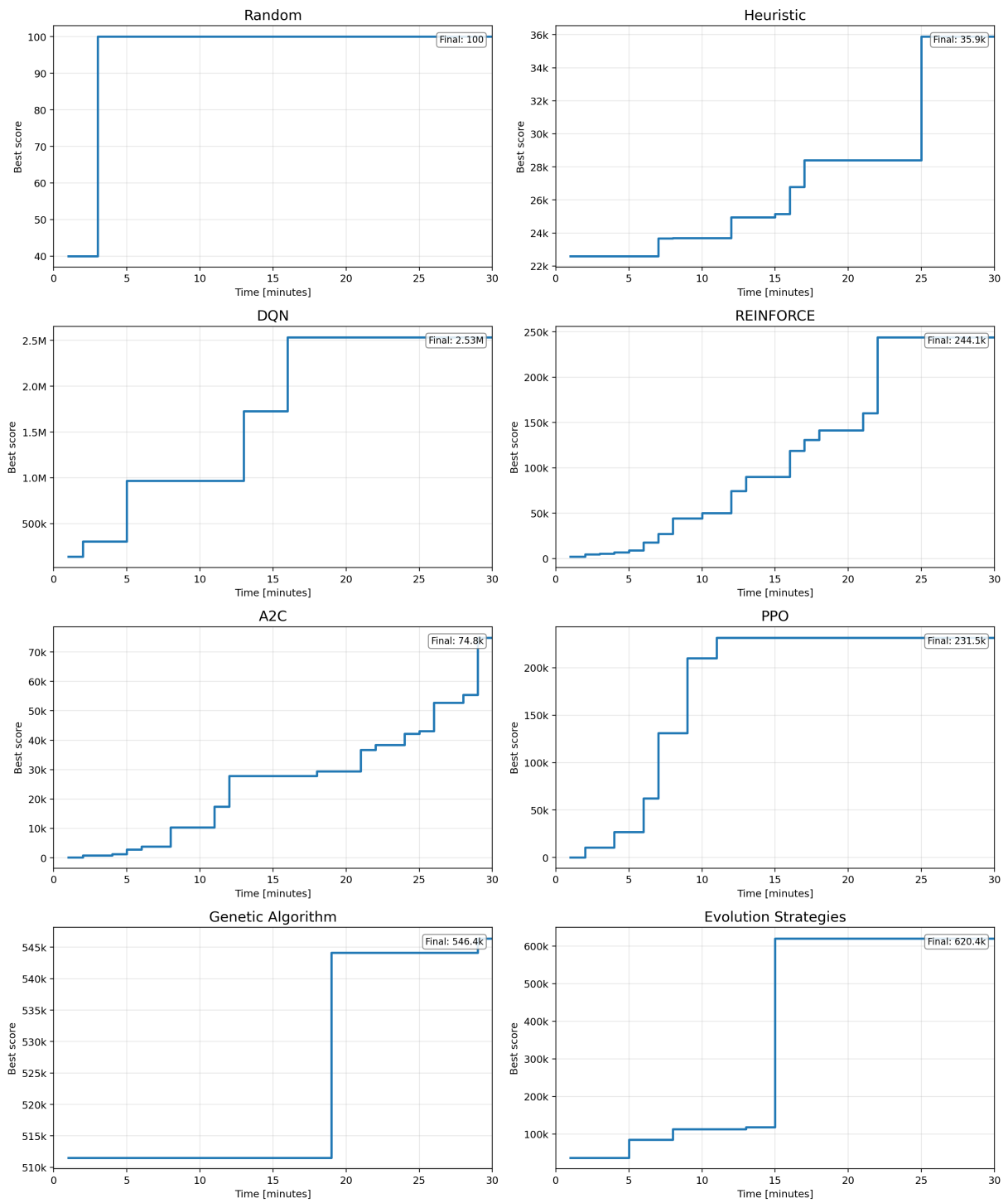
Rysunek 1 pokazuje najlepszy osiągnięty wynik w funkcji czasu dla wszystkich metod. Widać na nim, że DQN bardzo szybko odjechał pozostałym podejściom. W grupie metod optymalizacyjnych najlepszy chwilowy wynik w czasie 30 minut uzyskało ES, natomiast w późniejszej ewaluacji średniej jakości rozgrywek lepiej wypadł GA.



Rysunek 1: Najlepszy wynik osiągnięty w czasie 30 minut dla wszystkich metod.

Rysunek 2 pokazuje te same dane osobno dla każdego agenta. Dzięki temu lepiej widać tempo poprawy metod słabszych od DQN, zwłaszcza Heuristic, REINFORCE, PPO, GA i ES.

Training Progress by Agent



Rysunek 2: Postęp uczenia / optymalizacji osobno dla każdego agenta.

5.2 Ewaluacja końcowa po 10 grach

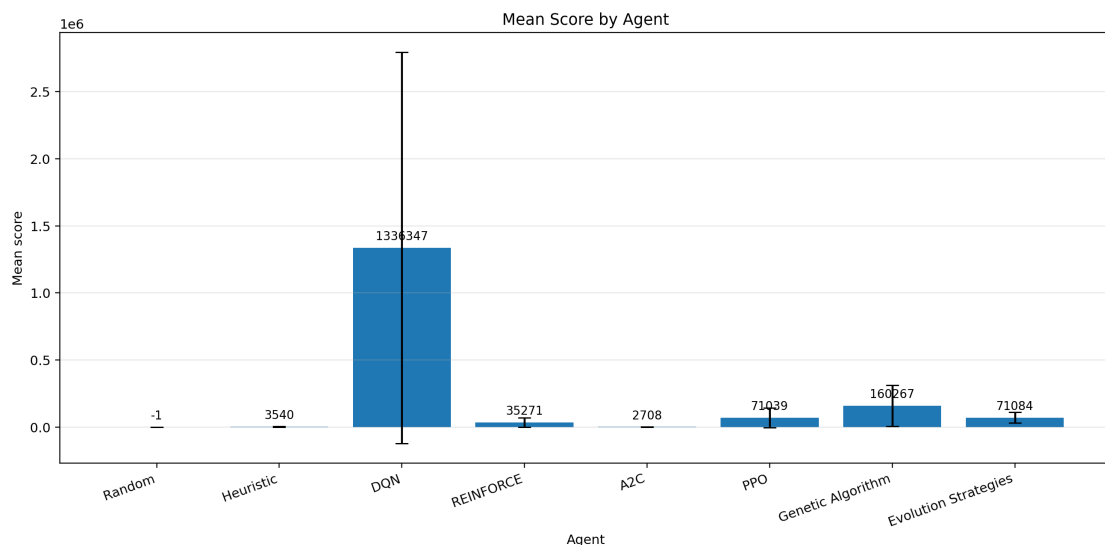
Tabela 1: Wyniki końcowej ewaluacji (10 gier na agenta).

Agent	Gry	Średni wynik	Mediana	Odch. std.	Najlepszy	Najgorszy	Śr. linii
Random	10	-0,6	0	1,0	0	-2	0
Heuristic	10	3 540,4	2 509	2 677,3	9 938	1 060	38,4
DQN	10	1 336 347,2	856 799	1 457 405,2	4 457 000	40 720	20 373,6
REINFORCE	10	37 195,8	30 510	29 448,6	91 300	3 638	777,2
A2C	10	999,2	939	810,0	2 040	40	8
PPO	10	53 778,8	30 088	49 094,4	127 778	8 918	1 103,7
Genetic Algorithm	10	201 583,2	190 830	142 601,8	452 918	21 738	4 552,7
Evolution Strategies	10	71 083,8	51 960	40 043,2	128 100	27 340	1 539,1

Na podstawie tabeli 1 można zauważyć, że:

- **DQN** osiągnął zdecydowanie najlepszy średni wynik: 1 336 347,2.
- **Genetic Algorithm** był najlepszą metodą optymalizacyjną ze średnim wynikiem 201 583,2.
- **PPO** i **REINFORCE** nauczyły się sensownej gry, ale były wyraźnie słabsze od DQN oraz GA.
- **A2C** w zadanym budżecie czasu praktycznie nie nauczył się gry na poziomie porównywalnym z PPO czy REINFORCE.
- **Heuristic** zdecydowanie przewyższył agenta losowego, co potwierdza, że ręcznie dobrane reguły dają sensowne zachowanie, ale ich sufit jakości jest znacznie niższy niż w przypadku najlepszych metod uczących się.

Rysunek 3 pokazuje średnie wyniki wraz z odchyleniem standardowym. Bardzo duży rozrzut DQN oznacza, że choć metoda ta potrafi osiągać spektakularne wyniki, to zdarzają jej się również bardzo wczesne przegrane, co z całą pewnością stanowi punkt wyjścia do późniejszej poprawy.



Rysunek 3: Średni wynik dla każdego agenta wraz z odchyleniem standardowym.

5.3 Porównanie trzech głównych podejść

W tabeli 2 porówniono najlepszych przedstawicieli każdej klasy na podstawie końcowej ewaluacji.

Tabela 2: Porównanie trzech podejść – najlepsi przedstawiciele każdej klasy.

Podejście	Reprezentant	Średni wynik	Mediana	Najlepszy wynik
Heurystyczne	Heuristic	3 540,4	2 509	9 938
Uczenie ze wzmocnieniem	DQN	1 336 347,2	856 799	4 457 000
Optymalizacja	Genetic Algorithm	201 583,2	190 830	452 918

Wniosek ilościowy jest jednoznaczny: **najlepsze wyniki uzyskało uczenie ze wzmocnieniem, a konkretnie DQN**. Metody optymalizacyjne także dawały bardzo dobre rezultaty i wyraźnie przewyższały heurystykę, jednak nie osiągnęły poziomu DQN. Heurystyka była natomiast wielokrotnie lepsza od agenta losowego i stanowiła sensowny punkt odniesienia.

6 Wyniki jakościowe

Oprócz samych liczb przeanalizowano zapisane GIF-y oraz zachowanie botów w trakcie rozgrywek.

6.1 Random

Agent losowy wykonuje ruchy bez żadnej strategii. Bardzo szybko tworzy dziury i wysokie kolumny, przez co przegrywa praktycznie natychmiast. Jest to poprawny punkt odniesienia, ale nie można mówić o żadnym sensownym zachowaniu.

6.2 Heuristic

Heurystyka gra znacznie lepiej od agenta losowego. Widać, że:

- dąży do wyrównywania planszy,
- ogranicza liczbę dziur,
- świadomie utrzymuje lewą studnię,
- potrafi przez pewien czas utrzymywać stabilną rozgrywkę.

Jednocześnie heurystyka jest mało elastyczna. Kiedy plansza przyjmuje nietypowy układ, ręcznie dobrana funkcja oceny nie zawsze potrafi dobrze zareagować.

6.3 DQN

DQN był jakościowo najbardziej imponujący. Agent potrafił:

- bardzo długo utrzymywać rozgrywkę,
- regularnie czyścić wiele linii,
- budować sensowne struktury pod przyszłe ruchy,
- wykorzystywać nadarżające się okazje dużo lepiej niż heurystyka.

Jednocześnie jego duża zmienność wskazuje, że nie każda partia przebiega równie dobrze. Innymi słowy: DQN potrafi grać wybitnie, ale nie zawsze stabilnie.

6.4 REINFORCE i PPO

REINFORCE oraz PPO nauczyły się zachowania wyraźnie lepszego od heurystyki. Widać u nich podstawową strategię budowania planszy i unikania oczywistych błędów. PPO zachowywał się dojrzałej od REINFORCE, ale oba agenty były wyraźnie słabsze od DQN.

6.5 A2C

A2C jakościowo wypadł słabo. W zadanym budżecie czasu nie udało się uzyskać stabilnej polityki. Agent czyścił niewiele linii i szybko przegrywał. To pokazuje, że nie każda poprawna teoretycznie metoda uczy się równie dobrze w danym środowisku i przy danym zestawie hiperparametrów.

6.6 GA i ES

Metody optymalizacyjne wykazywały bardziej zachowawcze, ale często całkiem solidne zachowanie. W porównaniu z PPO i REINFORCE wydają się mniej spektakularne w pojedynczych sekwencjach, ale umieją budować stabilne układy planszy. W końcowej ewaluacji lepiej wypadł GA, natomiast na wykresie najlepszego wyniku w czasie nieco wyżej doszedł ES.

7 Porównanie pracochłonności

Tabela 3: Porównanie pracochłonności trzech klas metod.

Podejście	Trudność implementacyjna	Koszt strojenia	Komentarz
Heurystyka	niska	niski	Najprostsze rozwiązanie; łatwe debugowanie i szybkie uruchomienie.
Uczenie ze wzmocnieniem	wysoka	bardzo wysoki	Najwięcej kodu i największa wrażliwość na hiperparametry.
Optymalizacja	średnia	średni	Prostsza od RL koncepcyjnie, ale kosztowna obliczeniowo.

Moim zdaniem:

- **heurystyka** była najszybsza do uruchomienia i najłatwiejsza do kontrolowania,
- **RL** było najbardziej wymagające, ale dało najlepszy wynik końcowy,
- **optymalizacja** okazała się dobrym kompromisem między jakością i trudnością, szczególnie w wariancie GA.

8 Wnioski końcowe

1. **Najskuteczniejszym agentem w projekcie był DQN.** Osiągnął najwyższy średni wynik i najwyższy pojedynczy wynik.
2. **Najlepszą metodą optymalizacyjną był algorytm genetyczny.** Choć ES uzyskał bardzo dobry przebieg uczenia, w końcowej ewaluacji GA dawał lepsze średnie wyniki.
3. **Heurystyka była zdecydowanie lepsza od losowości, ale dużo słabsza od najlepszych metod uczących się.** Mimo to była bardzo cenna jako punkt odniesienia i najłatwiejsza do przygotowania.

4. **A2C okazał się najsłabszą metodą uczącą się w ramach zadanego budżetu czasu.**
To pokazuje, że skuteczność metody zależy nie tylko od jej klasy teoretycznej, ale też od dopasowania do konkretnego problemu i strojenia.
5. **Z punktu widzenia kompromisu między pracochłonnością i jakością wyniki są następujące:** heurystyka jest najtańsza, optymalizacja jest dobrym kompromisem, a RL daje najwyższy potencjał kosztem największej złożoności.

Więcej informacji o projekcie, wraz z dodatkowymi materiałami wizualnymi, plikami wyników, GIF-ami oraz nagraniami z rozgrywek, znajduje się w repozytorium projektu:
<https://github.com/MNOWAK1234/PUTcourses/tree/main/GAI/Project1>.